

Signal and Image Processing

Assignment 3 - Fourier Transform

Group 8

Simone Skovgaard: wnf255

Noah Junge: qxk266

Marcus Friis-Hansen: lns611

February 2026

1 Fourier Transform - Theory

1.1

Prove that the continuous Fourier transform of a real and even function is real and even. Essentially, we would like to prove the following:

$$f_{\text{even}}(x) : \mathbb{R} \rightarrow \mathbb{R} \quad \mathcal{F}\{f_{\text{even}}\}(u) = \int_{-\infty}^{\infty} f_{\text{even}}(x) \cos(2\pi ux) dx$$

I will use this definition, where $f(x)$ is an even function:

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx = \int_{-\infty}^{\infty} f(x) (\cos(2\pi ux) - i \sin(2\pi ux)) dx^1$$

Split the integral into two parts:

$$\int_{-\infty}^{\infty} f(x) \cdot \cos(2\pi ux) dx - \int_{-\infty}^{\infty} f(x) \cdot i \cdot \sin(2\pi ux) dx$$

Now we use the fact that sinus is an odd function: ² $f_{\text{odd}} = \sin(x)$. That the product of an even and odd function is odd: $g_{\text{odd}}(x) = f_{\text{even}}(x) \cdot h_{\text{odd}}(x)$. And that the definite integral of an odd function is zero ³ $\int_{-\infty}^{\infty} f_{\text{odd}}(x) dx = 0$.

$$F(u) = \int_{-\infty}^{\infty} f(x) \cdot \cos(2\pi ux) dx - 0$$

$$F(u) = \int_{-\infty}^{\infty} f(x) \cdot \cos(2\pi ux) dx$$

This proves that it is real, since it contains no i . Now we will prove that it is even using this definition: A function is even if⁴ $f_{\text{even}}(x) = f_{\text{even}}(-x)$. I will show that $F(u) = F(-u)$

¹fourier II - slide 24

²Fourier II - slide 20

³Fourier II - slide 21

⁴Fourier II - slide 20

$$F(-u) = \int_{-\infty}^{\infty} f(x) \cdot \cos(2\pi(-u)x) dx$$

Here we remember that cosine is an even function⁵ $f_{\text{even}} = \cos(x)$, meaning $\cos(-u) = \cos(u)$, so:

$$\cos(2\pi(-u)x) = \cos(2\pi ux)$$

it then follows

$$F(-u) = \int_{-\infty}^{\infty} f(x) \cdot \cos(2\pi(-u)x) dx = \int_{-\infty}^{\infty} f(x) \cdot \cos(2\pi ux) dx = F(u)$$

which proves that it is even, concluding the proof.

1.2

Consider the box function

$$b_a(x) = \begin{cases} \frac{1}{a} & \text{if } |x| \leq \frac{a}{2} \\ 0 & \text{otherwise} \end{cases}$$

1.2.1

Show that $\int_{-\infty}^{\infty} b_a(x) dx = 1$

Since $|x| \leq \frac{a}{2}$, we put in this as a limit in the integral, and we know the value of $b_a(x)$ is $b_a(x) = \frac{1}{a}$ within these limits, so we also input that.

$$\int_{-\frac{a}{2}}^{\frac{a}{2}} \frac{1}{a} dx$$

We factor out the constant

$$\frac{1}{a} \int_{-\frac{a}{2}}^{\frac{a}{2}} 1 dx$$

We integrate and reduce it

$$\frac{1}{a} \left(\frac{a}{2} - \left(-\frac{a}{2} \right) \right) = \frac{1}{a} \left(\frac{a}{2} + \frac{a}{2} \right) = \frac{1}{a} \cdot \frac{2a}{2} = \frac{1}{a} \cdot \frac{2}{2} \cdot a = \frac{1}{a} \cdot 1 \cdot a = \frac{a}{a} = 1$$

1.2.2

Show that the continuous Fourier transform of b_a , using the definition of the Fourier transform given in Bracewell Chapter 2 (system 1), is $B_a(k) = \frac{1}{ak\pi} \sin(ak\pi)$. Rewrite $B_a(k)$ using the $\text{sinc}(x) = \frac{\sin x}{x}$

Below to the left is the definition of the continuous Fourier transform given in Bracewell, chapter 2, system 1.

$$\int_{-\infty}^{\infty} f(x) e^{-i2\pi xs} dx$$

⁵Fourier II - slide 20

Note that we will use k instead of s . We recognize that in this case $f(x) = b_a(x)$, and see that we need to prove:

$$\int_{-\infty}^{\infty} b_a(x) e^{-i2\pi xk} dx = B_a(k) = \text{sinc}(ak\pi)$$

we remind ourselves of the result from the earlier task, and factor out the constant:

$$\int_{-\frac{a}{2}}^{\frac{a}{2}} \frac{1}{a} \cdot e^{-i2\pi xk} dx = \frac{1}{a} \int_{-\frac{a}{2}}^{\frac{a}{2}} e^{-i2\pi xk} dx$$

We integrate

$$\frac{1}{a} \cdot \left[\frac{e^{-i2\pi xk}}{-i2\pi k} \right]_{-\frac{a}{2}}^{\frac{a}{2}} = \frac{1}{a} \cdot \frac{1}{-i2\pi k} \left[e^{-i2\pi xk} \right]_{-\frac{a}{2}}^{\frac{a}{2}} = \frac{1}{a} \cdot \frac{1}{-i2\pi k} \left(e^{-i2\pi(\frac{a}{2})k} - e^{-i2\pi(-\frac{a}{2})k} \right) = \frac{1}{a} \cdot \frac{1}{-i2\pi k} (e^{-i\pi ak} - e^{i\pi ak})$$

We now use Euler's formula⁶ $e^{i\theta} = \cos\theta + i\sin\theta$. And from that it follows: $e^{-i\theta} - e^{i\theta} = (\cos\theta - i\sin\theta) - (\cos\theta + i\sin\theta) = -2 \cdot i\sin\theta$

$$\frac{1}{a} \cdot \frac{1}{-i2\pi k} (e^{-i\pi ak} - e^{i\pi ak}) = \frac{1}{a} \cdot \frac{1}{-i2\pi k} (-2)i\sin\pi ak$$

We see that $-2i$ cancels out, and simplify it further

$$\frac{1}{a} \cdot \frac{1}{-i2\pi k} (-2)i\sin\pi ak = \frac{1}{a} \cdot \frac{-2i}{-i2\pi k} \sin\pi ak = \frac{1}{a} \cdot \frac{\sin\pi ak}{\pi k} = \frac{\sin\pi ak}{a\pi k}$$

$$B_a(k) = \frac{\sin\pi ak}{a\pi k}$$

And that was exactly what we needed to prove. Furthermore, we are asked to rewrite $B_a(k)$ like shown below, using $\text{sinc}(x) = \frac{\sin x}{x}$:

$$B_a(k) = \frac{1}{ak\pi} \sin(ak\pi) = \frac{\sin(ak\pi)}{ak\pi} = \text{sinc}(ak\pi)$$

1.2.3

Show that $\lim_{a \rightarrow 0} B_a(k) = 1$, (**Hint:** $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$)

We just proved that $B_a(k) = \frac{\sin(ak\pi)}{ak\pi}$. So if we use the hint:

$$\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$$

Then it follows that

$$\lim_{a \rightarrow 0} B_a(k) = \lim_{a \rightarrow 0} \frac{\sin ak\pi}{ak\pi} = 1$$

1.2.4

When a is near zero, then b_a is narrow in the x space domain. In this case, would you consider its Fourier transform narrow or wide in the frequency domain, k ? What is the relation when a is large? Explain your answer.

This is described in Bracewell⁷, where it is stated that compression in the time(space)

⁶Kalkulus 4. edition, Tom Lindstrøm. p.135 or Fourier II - slide 13

⁷Bracewell, pp. 101 - 103

domain corresponds to expansion in the frequency domain, and vice versa. This means that when a is near zero and b_a is narrow in the x space domain, then its Fourier transform is wide in the frequency domain k . It follows that when a is large, then b_a is wide in the space domain x , which makes its Fourier transform contract horizontally and grow vertically in the frequency domain k , such that the area beneath the Fourier transform remains constant.

2 Fourier Transform – In Practice

2.1

Given a grayscale image stored as a `float64` array, we compute the 2D FFT and define the power spectrum as $P(u) = |F(u)|^2$. The zero frequency is centered using `fftshift`, and the spectrum is displayed using $\log(1 + P)$ scaling to compress its dynamic range.

```
F = fft2(image)
F_shifted = fftshift(F)
power_spectrum = np.abs(F_shifted)**2
```

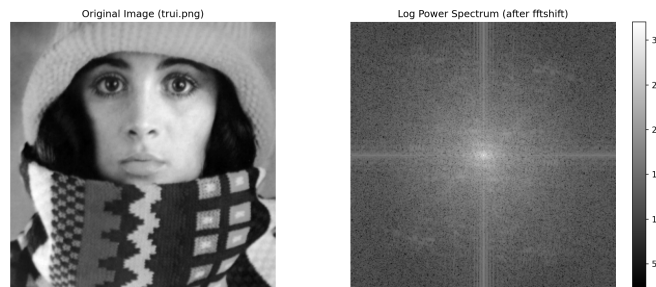


Figure 1: Original image and its log-scaled power spectrum.

The central bright peak corresponds to the DC component (mean intensity). Low spatial frequencies are concentrated near the center, while higher frequencies appear further away. The faint cross pattern reflects dominant horizontal and vertical edges in the image.

2.2

We add a planar cosine wave to `cameraman.tif` with $a_0 = 100$, $v_0 = 30$, $w_0 = 50$:

```
cosine_wave = a0*np.cos(2*np.pi*(v0*X/N + w0*Y/M))
img_noisy = img + cosine_wave
```

The cosine introduces diagonal stripes in the spatial domain and produces two symmetric impulse peaks at $(\pm v_0, \pm w_0)$ in the frequency domain.

To remove them, we design a notch filter (radius = 5 pixels):

```
notch_filter = np.ones((M,N))
for py, px in [peak1, peak2]:
    dist = np.sqrt((YY-py)**2 + (XX-px)**2)
    notch_filter[dist <= notch_radius] = 0.0
F_filtered = F_noisy * notch_filter
img_filtered = np.real(iff2(iffshift(F_filtered)))
```



Figure 2: Original, noisy, filtered spectra and reconstructed image.

After filtering, the impulse peaks disappear and the stripes are almost completely removed. Small differences remain due to the removed frequency neighborhood.

2.3

The radial average computes the mean power at each distance r from the spectrum center, producing a 1D representation of energy versus spatial frequency.

```
def radial_average_power_spectrum(image):
    ps = power_spectrum(image)
    M, N = ps.shape
    cy, cx = M//2, N//2
    y, x = np.ogrid[:M, :N]
    dist = np.sqrt((y-cy)**2 + (x-cx)**2)
    dist_int = np.round(dist).astype(int)
    max_dist = min(cy, cx)
    avg_power = np.zeros(max_dist+1)
    for r in range(max_dist+1):
        mask = dist_int == r
        if np.any(mask):
            avg_power[r] = np.mean(ps[mask])
    return np.arange(max_dist+1), avg_power
```

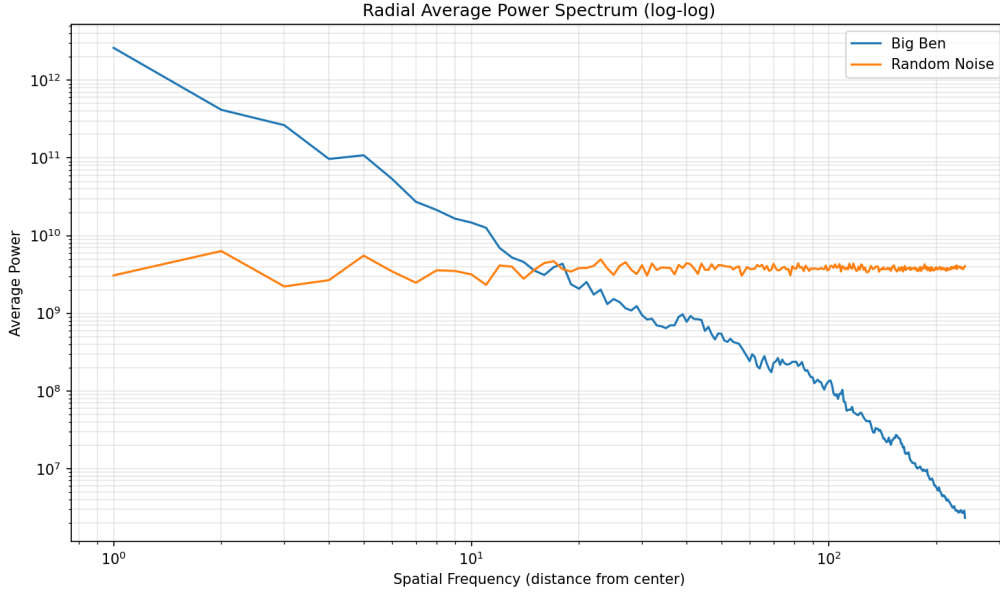


Figure 3: Radial average power spectrum (log-log).

- **Big Ben:** Power decreases approximately linearly in the log-log plot, consistent with a $1/f^2$ behavior typical of natural images.
- **Random noise:** The spectrum is nearly flat, indicating equal contribution from all spatial frequencies.

2.4

We average the power within angular bins of 10° in the spatial frequency range $[10, 100]$.

```
def angular_average_power_spectrum(image, freq_range=(10,100),
    angle_bin_size=10):
    ps = power_spectrum(image)
    M, N = ps.shape
    cy, cx = M//2, N//2
    y, x = np.ogrid[:M, :N]
    dist = np.sqrt((y-cy)**2 + (x-cx)**2)
    angle = np.degrees(np.arctan2(y-cy, x-cx)) % 360
    freq_mask = (dist>=freq_range[0]) & (dist<=freq_range[1])
    avg_power = []
    for a in range(0,360,angle_bin_size):
        mask = freq_mask & (angle>=a) & (angle<a+angle_bin_size)
        avg_power.append(np.mean(ps[mask]) if np.any(mask) else 0)
    return np.arange(0,360,angle_bin_size), np.array(avg_power)
```

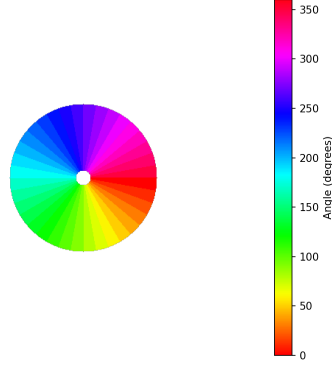


Figure 4: Angular bin configuration.

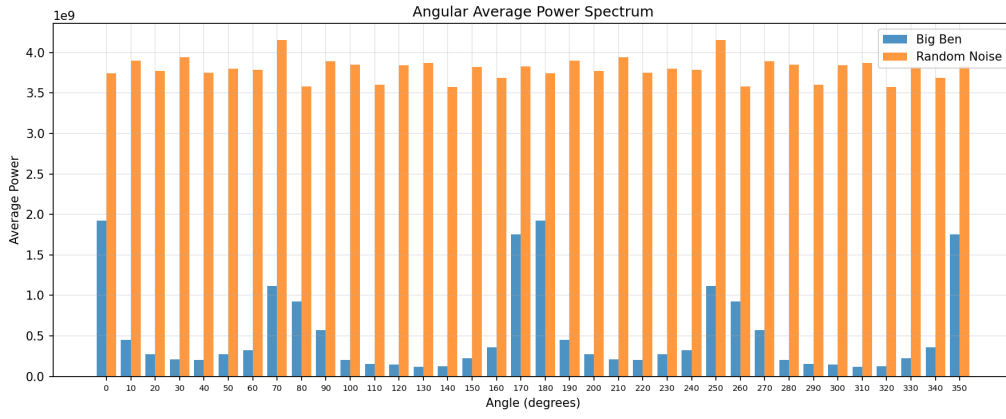


Figure 5: Angular average power spectrum.

- **Big Ben:** Peaks near 0°/180° and 90°/270° indicate dominant vertical and horizontal structures.
- **Random noise:** Nearly uniform distribution across angles.

2.5

Using the differentiation property

$$\mathcal{F} \left\{ \frac{\partial^n f}{\partial x^n} \right\} = (i2\pi k_x)^n F(k_x, k_y),$$

spatial derivatives are computed via multiplication in the frequency domain.

```
F = fft2(image)
kx = fftfreq(N, d=1.0)
ky = fftfreq(M, d=1.0)
KX, KY = np.meshgrid(kx, ky)
H = (1j*2*np.pi*KX)**dx_order * (1j*2*np.pi*KY)**dy_order
result = np.real(iff2(F*H))
```

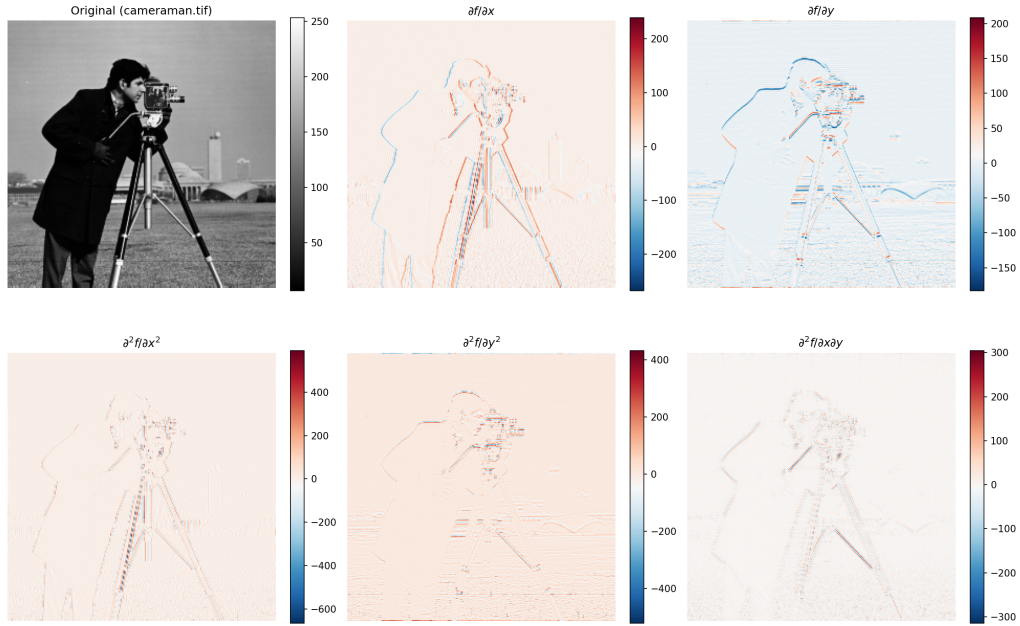


Figure 6: Spatial derivatives computed in the frequency domain.

First-order derivatives highlight directional edges: $\partial f / \partial x$ emphasizes vertical edges and $\partial f / \partial y$ horizontal edges.

Second-order derivatives enhance fine structures, while the mixed derivative $\partial^2 f / \partial x \partial y$ highlights diagonal features.

Assuming unit spacing ($\Delta x = \Delta y = 1$), derivatives are computed efficiently via element-wise multiplication.